

School of Mathematics and Statistics, UNSW
MATH3311/MATH5335 Python
Term 2, 2023

Exercises 7: Implied Volatility and Nonlinear Equations

1. On 27-Aug-2014 a NAB share was \$34.81. Strikes and prices for some NAB options with expiry 25-Sep-2014 were (<http://www.asx.com.au/asx/markets/optionPrices.do>)

Strike X (\$)	33.5	34	34.5	35	35.5	36
Call c_{mkt} (\$)	1.460	1.040	0.640	0.370	0.155	0.070

Assume a risk-free interest rate of 2.5%, and a 365 day year basis. Under the assumptions of the Black and Scholes model the implied volatility is the same for all strikes. Download the file `blackscholes.py` from Moodle. The `date` function from the `datetime` module can be used to calculate time differences between dates. After doing

```
t0 = date.fromisoformat('2014-08-27')
tf = date.fromisoformat('2014-09-25')
```

the expression `(tf-t0).days` gives the number of days between `t0` and `tf`.

- (a) Use the Python function `root_scalar` from `scipy.optimize` (This function can be used to find a zero of a scalar function.) to find the implied volatility for each option.
 - (b) Check that the calculated implied volatility does give the quoted market value.
 - (c) Plot the implied volatility against the strike, and comment on your results.
2. The Black & Scholes value p at time t for a put with expiry T , strike X , asset price S and risk-free rate r is

$$p(\sigma) = Xe^{-r(T-t)}N(-d_2) - SN(-d_1) \quad (1)$$

where d_1 and d_2 are given by

$$d_1 = \frac{\log(S/X) + r(T-t)}{\sigma\sqrt{T-t}} + \frac{1}{2}\sigma\sqrt{T-t},$$
$$d_2 = \frac{\log(S/X) + r(T-t)}{\sigma\sqrt{T-t}} - \frac{1}{2}\sigma\sqrt{T-t} = d_1 - \sigma\sqrt{T-t}.$$

- (a) Find the limit p_0 of $p(\sigma)$ as $\sigma \rightarrow 0^+$.
- (b) Show that $p'(\sigma) > 0$ for all $\sigma > 0$.
- (c) Show that if the market price p_{mkt} is larger than p_0 then there exists a unique implied volatility σ_{put} .
- (d) Plot the value of the put as a function of σ . Hint: you can modify `blackscholes.m`.

3. The standard normal cumulative probability function $\Phi : \mathbb{R} \rightarrow [0, 1]$ (Finance texts often use $N(x)$ instead of $\Phi(x)$) is

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{u^2}{2}} du.$$

The inverse of the standard cumulative normal probability function is $\Phi^{-1} : [0, 1] \rightarrow \mathbb{R}$ such that

$$\Phi^{-1}(p) = x \quad \text{where} \quad p = \Phi(x).$$

- (a) For a given $p \in [0, 1]$, pose the problem of finding $\Phi^{-1}(p)$ as the solution of a nonlinear equation $f(x) = 0$.
 - (b) Calculate the derivatives $f'(x)$ and $f''(x)$.
 - (c) Prove that x is a simple root of f .
 - (d) If $\bar{x}$ is an approximate value of the root x , with error $|\bar{x} - x| \approx 10^{-6}$, estimate the error after one iteration of the following methods:
 - i. a linearly convergent method with rate $\beta = 0.9$;
 - ii. the Secant method
 - iii. Newton's method
 - iv. Halley's third order rational method
4. The productivity P of a computer software project depends on the size L of the project and the maximum staffing level S . A model of the form

$$P = x_1 L^{x_2} S^{x_3} \tag{2}$$

is assumed, where x_1 , x_2 , and x_3 are parameters to be determined. Data obtained from past projects is

Project i	1	2	3	4	5
Productivity P_i	450	830	547	638	1508
Project size L_i	3539	42487	12620	21698	30521
Staffing level S_i	3	7	5	6	4

Table 1: Productivity of computer software projects

- (a) How many of the data values are required to determine the parameters? Pick a subset **I** of the data with the correct number of elements. Define **P**, **L**, **S** as NumPy arrays with `dtype=float64`, and take **I**=[0, 1, 2].
- (b) Write a Python function `softprod` to calculate the residual for the system of equations corresponding to your subset **I**. You will need to pass a value for the optional argument `jac`. You can use the optional argument `args` to pass the data **L**, **S**, **P** and `jac` through to `softprod`.
- (c) Use the Python function `fsolve` to find the parameter values. A reasonable starting value is $x_1 = 5$, $x_2 = 1$, $x_3 = -1$.

- (d) Modify your function `softprod` to also calculate the Jacobian if it is called with more than one output argument. Change the options to `fsolve` so the Jacobian is used and do a finite difference check of the derivatives. The `root` function from `scipy.optimize` is used to solve.
- (e) Are the final parameter values a solution to the system of equations? Add a boolean argument `jac` for `softprod`. If `True`, then `softprod` returns the residual `r` and the Jacobian `J`. If `False`, then return only the residual.
- (f) Is the Jacobian nonsingular at the solution?
- (g) What rate of convergence are you getting from `fsolve`?
- (h) Try different subsets `I` of the data to determine the parameters, and see if this makes much difference to the parameter estimates produced. You can loop of all possible choices for `I` as follows.

```

selections = [ [0,1,2], [0,2,3], [0,2,4], [1,2,3], [1,2,4], [2,3,4] ]
for I in selections:
    ...

```

- (i) Use the Python function `lsqnonlin` from the `scipy.optimize` to find the best least squares fit to all the data, and compare the parameter estimates obtained with those obtained using `fsolve`.
- (j) Find a transformation of the problem that produces a linear problem, and find the corresponding least squares estimates of the parameters. Use the `lstsq` function from the `scipy.linalg`.
- (k) Comment on the significance of the parameters, in particular x_3 .

A general reference for interest is F. P. Brooks Jr., *The Mythical Man-Month: Essays on Software Engineering* and http://en.wikipedia.org/wiki/The_Mythical_Man-Month.